# Laboratório de Concorrência · Aula 10

BRUNO VINICIUS COUTINHO MIANTE DA SILVA · RA 241516052 · Turma ESOFT C
Gerado em 20/10/2026, 19:30:53 · tempo registrado no cronômetro: 0:00

## Preparação: Concorrência em cinco minutos

**0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.**

Na cooperação, uma tarefa depende da outra para chegar ao objetivo; por exemplo, numa mudança, primeiro alguém encaixota os objetos e depois outra pessoa os transporta. Na competição, duas pessoas tentam usar o mesmo recurso limitado, como dois colegas querendo utilizar a única impressora ao mesmo tempo.

## Prática 1: A condição de corrida

**Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?**

Espero que o resultado seja 2.000.000, pois são duas threads incrementando o contador um milhão de vezes cada.

**1ª execução (n = 1.000.000)**

1518837

**2ª execução (n = 1.000.000)**

1499436

**3ª execução (n = 1.000.000)**

1593359

**Saída do programa**

```
Execuções com n = 1.000.000: 1518837; 1499436; 1593359.
Com n = 1000: 2000.
```

**1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?**

Não. O valor esperado era 2.000.000, mas as três execuções terminaram com valores menores e diferentes. O incremento envolve ler, somar e gravar; como essas etapas não são atômicas, as threads podem interferir uma na outra e perder atualizações.

**1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.**

Suponha que o contador esteja em 10. A thread A lê 10 e calcula 11. Antes de A gravar, B também lê 10 e calcula 11. A grava 11 e depois B grava 11 sobre o mesmo valor. O resultado final fica em 11, embora duas atualizações devessem produzir 12.

**1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.**

Com n = 1000, o resultado observado foi 2000, que é o esperado. Isso não significa que a corrida desapareceu: numa execução curta, a intercalação pode não causar perdas. Em sistemas reais, o defeito pode aparecer apenas sob maior carga e gerar dados incorretos.

## Prática 2: Todas as respostas possíveis (Figura 13.1)

**Ordens testadas no simulador**

*não respondido*

**Saída do programa**

```
O simulador explora diferentes intercalações das operações das threads. Os resultados
possíveis indicados na atividade são 4, 6, 7 e 8.
```

**2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?**

Há 20 ordens possíveis no simulador e quatro resultados diferentes: 4, 6, 7 e 8. O resultado correto é 8; apenas uma das ordens apresentadas chega a esse valor.

**2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?**

Uma ordem que termina em 4 permite que as duas threads leiam valores antigos antes de gravar. Uma delas grava o resultado calculado e a outra sobrescreve essa atualização com um valor também calculado a partir do estado anterior. Assim, uma operação de B deixa de aparecer no valor final.

## Prática 3: Corrigindo a corrida

**Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?**

Acredito que a versão sem sincronização será a mais rápida, mas poderá apresentar um valor incorreto. synchronized e AtomicInteger devem proteger as atualizações e chegar ao resultado correto de 2.000.000.

**Saída do programa**

```
Sem sincronização: 1.955.740 (4 ms)
AtomicInteger: 2.000.000 (17 ms)
synchronized: 2.000.000 (95 ms)
```

**3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?**

As versões synchronized e AtomicInteger chegaram a 2.000.000. A versão sem proteção terminou em 1.955.740. Nos tempos observados, a versão sem sincronização levou 4 ms, AtomicInteger 17 ms e synchronized 95 ms. A coordenação tem custo, mas impede que atualizações concorrentes sejam perdidas.

**3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?**

Não. Uma transferência precisa debitar uma conta e creditar outra como uma única operação lógica. Mesmo com cada saldo atômico, pode haver uma interrupção entre o débito e o crédito. É preciso proteger a transação completa e garantir consistência, por exemplo com transação no banco de dados e bloqueios adequados.

## Prática 4: Impasse (deadlock)

**Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?**

Não necessariamente. Se A mantiver X e esperar Y enquanto B mantiver Y e esperar X, cada uma ficará esperando um recurso preso pela outra, formando um deadlock.

**Saídas (antes e depois da correção)**

```
Antes da correção: as threads ficam aguardando e as mensagens de conclusão não aparecem.
Depois da correção: ambas conseguem terminar quando adquirem os recursos em uma ordem consistente.
```

**4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.**

A thread A adquiriu X e ficou esperando Y. A thread B adquiriu Y e passou a esperar X. Como cada uma segura o recurso que a outra precisa, nenhuma consegue avançar nem liberar o recurso, causando o impasse.

**4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.**

Se todas as threads solicitarem os recursos na mesma ordem, não se forma o ciclo de espera em que cada uma segura um recurso e aguarda o outro. Isso pode ser aplicado, por exemplo, a transferências entre duas contas bancárias: bloquear as contas sempre pela mesma ordem reduz o risco de deadlock.

## Prática 5: Mensagens em vez de memória compartilhada

**Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?**

Com seis tarefas de 100 ms, espero aproximadamente 600 ms com um trabalhador, 200 ms com três trabalhadores e 100 ms com seis, sem contar o custo de coordenação.

**Tempo com 1 trabalhador**

600 ms

**Tempo com 3 trabalhadores**

200 ms

**Tempo com 6 trabalhadores**

100 ms

**Saída do programa**

```
Estimativas: 1 trabalhador ≈ 600 ms; 3 trabalhadores ≈ 200 ms; 6 trabalhadores ≈ 100 ms. Os valores reais podem variar conforme o escalonamento e o custo de execução.
```

**5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?**

Um trabalhador executa as seis tarefas em sequência, levando cerca de 600 ms. Três trabalhadores podem dividir as tarefas e terminar em aproximadamente 200 ms; seis podem executá-las em paralelo, em cerca de 100 ms. O exemplo usa canais para distribuir tarefas e não faz várias threads modificarem diretamente a mesma variável compartilhada.

**Saídas (antes e depois da troca)**

```
Canal sem buffer: o envio espera até que alguém receba a mensagem.
Canal com buffer de capacidade 1: a mensagem pode ficar armazenada até ser recebida.
```

**5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)**

O primeiro programa pode travar porque, num canal sem buffer, quem envia fica bloqueado até existir um receptor pronto. Se não houver recebimento naquele momento, o programa não avança. Um buffer com espaço para uma mensagem permite armazená-la temporariamente e concluir o envio.

## Prática 6: O compilador como guarda

**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

Espero que o compilador Rust rejeite uma tentativa de alterar de forma insegura uma variável compartilhada entre threads, pois as regras de propriedade e segurança impedem vários acessos incompatíveis.

**Saídas (o erro e o resultado com move)**

```
Sem compartilhamento seguro, Rust acusa erro de compilação.
Com move, a thread recebe o valor capturado e o exemplo imprime 0 na variável original de
main.
```

**Saída do programa**

```
A thread trabalha com o valor capturado por move; a variável original continua com o valor 0.
```

**6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)**

O uso de move transfere para a thread o valor capturado. Como o contador é um número simples, a thread recebe uma cópia e altera essa cópia, não a variável de main. Por isso, ao imprimir a variável original, o resultado continua sendo 0.

**6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?**

No exemplo Java, a falha pode ser percebida durante a execução, quando o contador termina com valor menor que o esperado. Rust detecta algumas formas de compartilhamento inseguro já na compilação. Isso previne erros antes de executar, mas exige conhecer e usar corretamente ferramentas como Arc e Mutex quando o compartilhamento é necessário.

## Prática 7: Uma thread só: o laço de eventos

**Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?**

A ordem esperada é A, D, C, B: primeiro roda o código síncrono; depois a continuação da promessa; por fim, o callback do temporizador.

**Saída do programa**

```
A
D
C
B
```

**7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?**

A saída foi A, D, C, B. O código síncrono imprime A e D primeiro. A continuação da promessa é executada antes do callback do temporizador. Por isso, mesmo com tempo zero, B não aparece imediatamente.

**7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?**

O cálculo pesado bloqueia a thread principal por cerca de 300 ms e impede o event loop de tratar o timer no prazo previsto. Em um servidor Node.js, esse tipo de bloqueio atrasa outras requisições; tarefas pesadas devem ser divididas ou executadas fora da thread principal.

## Fechamento: Quem impede a corrida?

**Java sem sincronização**

Ninguém: há corrida

**Java com synchronized**

O programador, com sincronização

**Go com canais**

O projeto: não há memória compartilhada

**Rust com Arc e Mutex**

O compilador

**JavaScript**

Só existe uma thread

**8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?**

Pode acontecer uma condição de corrida: as duas pessoas verificam que ainda há um ingresso e ambas concluem a compra, vendendo duas vezes o mesmo lugar. A consulta e a reserva precisam ser uma única operação atômica, protegida por transação e controle de concorrência no banco de dados.

**8.2 Em uma frase: o que mais surpreendeu você nesta atividade?**

O que mais me chamou a atenção foi que até uma soma simples pode produzir um resultado errado quando várias threads atualizam o mesmo dado sem coordenação.